SOFTWARE PROJECT LAB

KidScribe

A Tablet App that Teaches Children Aged 5-8 to Write by Hand



Software Project Lab - Group 18

Team 18

02

What KidScribe Does

Real handwriting, not only tapping

KidScribe helps children practice real handwriting with a tablet pen.

Children trace and write letters instead of only tapping them. The app checks their handwriting and gives simple feedback on what is correct and what needs improvement.



trace -> write -> check -> improve

03

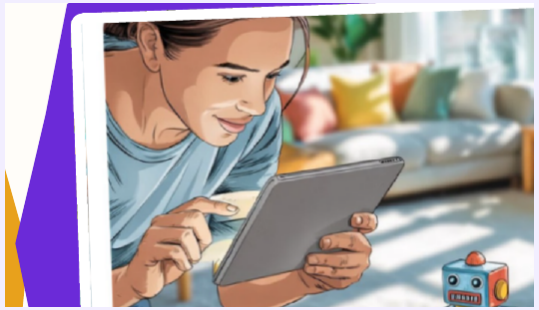
The People It Serves

KidScribe has three main users.



Children

Children write all 26 English letters, practice common words, and follow five learning stages. The app checks their writing and gives instant feedback.



Parents

Create and manage child profiles. They can see weekly progress, completed letters and words, and weak areas.



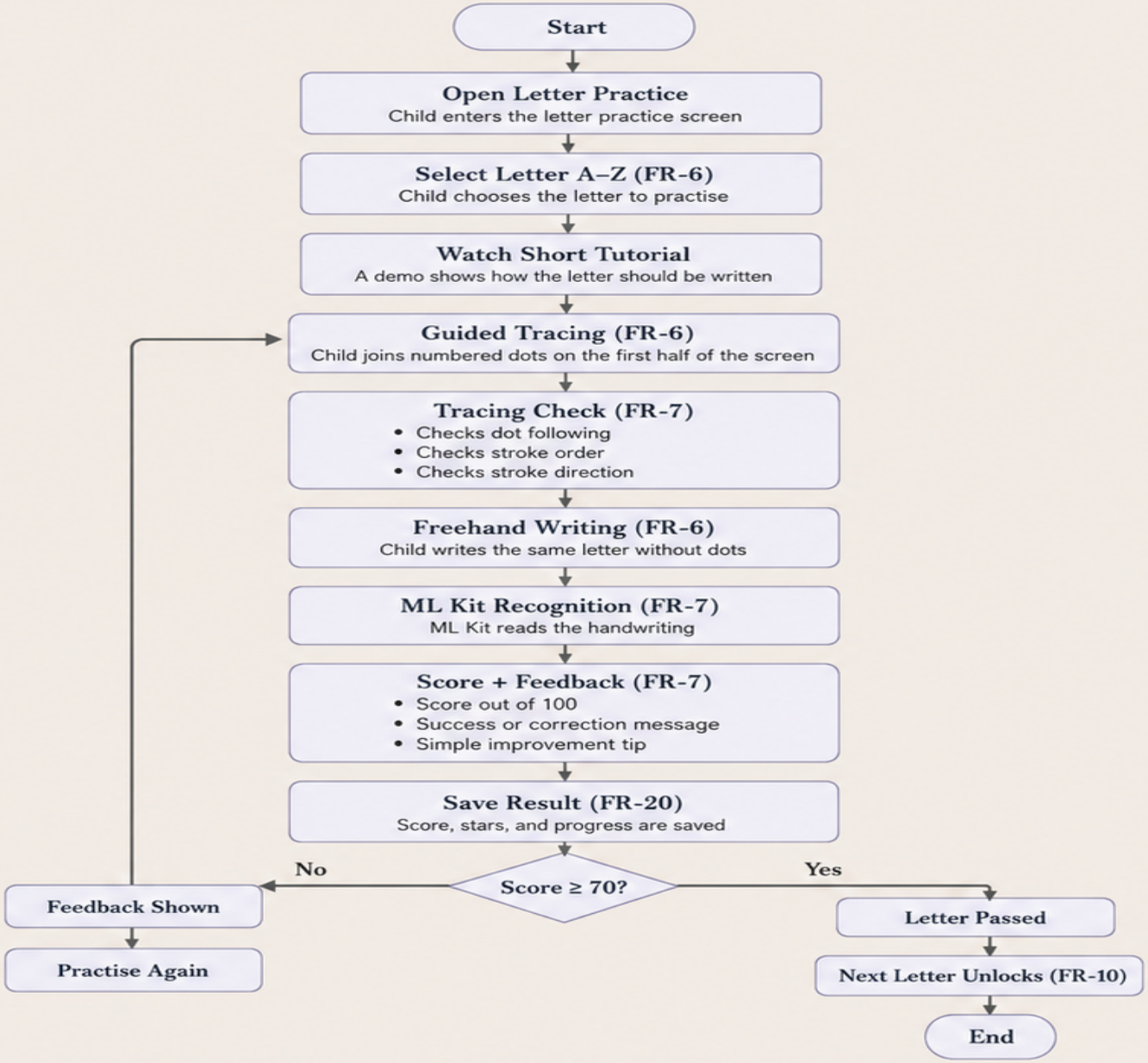
Teachers

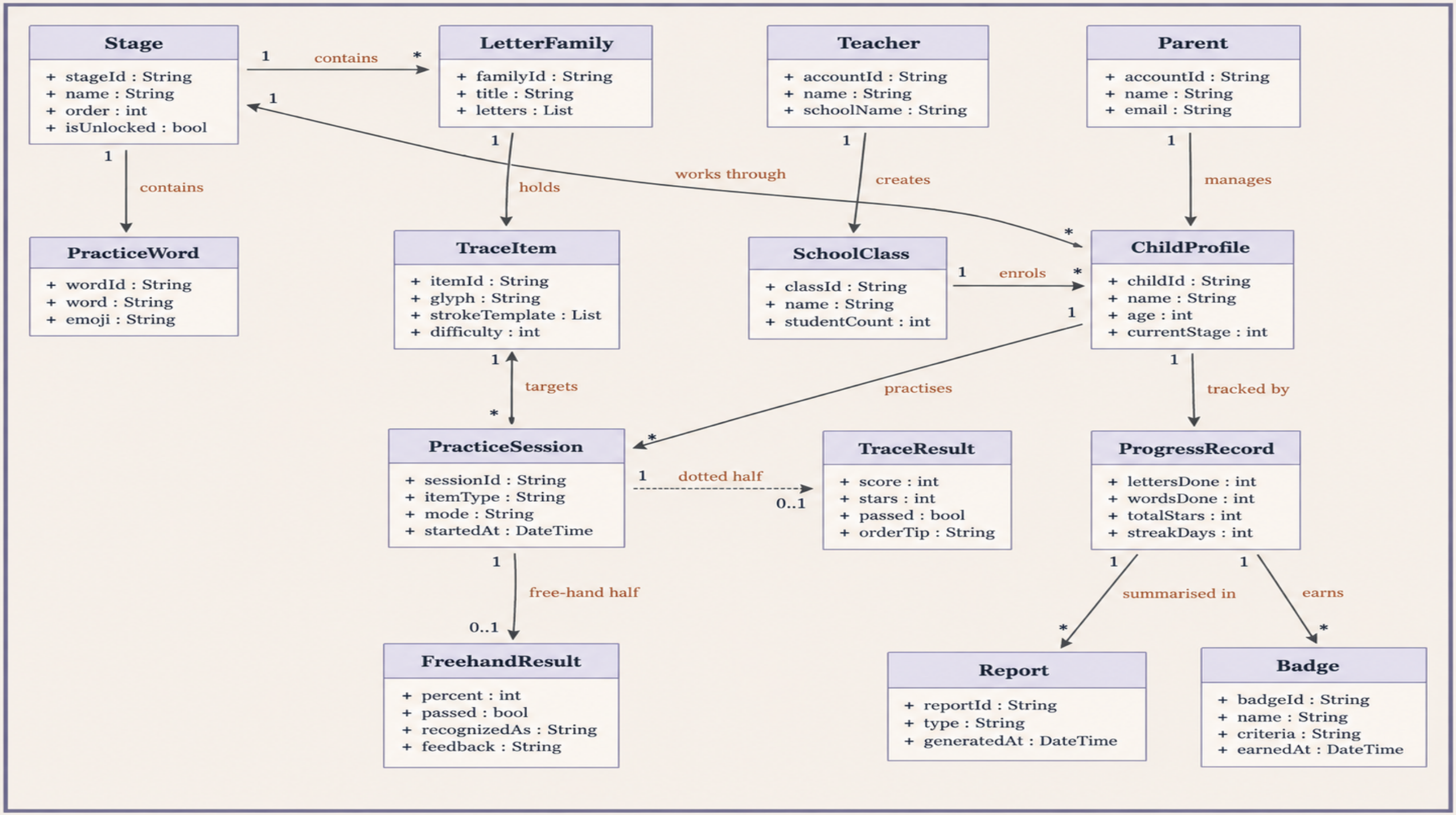
Create classes, add students, view student progress, identify students who are behind, and save reports.

To keep children motivated, the app gives stars, badges, rewards, and daily streaks.

Related SRS: FR-3 to FR-10, FR-14, FR-18, FR-19.

Letter Practice Workflow




Stage
+ stageId : String
+ name : String
+ order : int
+ isUnlocked : bool
1
contains
*
LetterFamily
+ familyId : String
+ title : String
+ letters : List
Teacher
+ accountId : String
+ name : String
+ schoolName : String
Parent
+ accountId : String
+ name : String
+ email : String
1
contains
PracticeWord
+ wordId : String
+ word : String
+ emoji : String
1
holds
works through
creates
manages
*
TraceItem
+ itemId : String
+ glyph : String
+ strokeTemplate : List
+ difficulty : int
SchoolClass
+ classId : String
+ name : String
+ studentCount : int
1
enrols
*
ChildProfile
+ childId : String
+ name : String
+ age : int
+ currentStage : int
1
targets
*
practises
tracked by
*
PracticeSession
+ sessionId : String
+ itemType : String
+ mode : String
+ startedAt : DateTime
1
dotted half
0..1
TraceResult
+ score : int
+ stars : int
+ passed : bool
+ orderTip : String
ProgressRecord
+ lettersDone : int
+ wordsDone : int
+ totalStars : int
+ streakDays : int
1
free-hand half
0..1
summarised in
earns
*
FreehandResult
+ percent : int
+ passed : bool
+ recognizedAs : String
+ feedback : String
Report
+ reportId : String
+ type : String
+ generatedAt : DateTime
Badge
+ badgeId : String
+ name : String
+ criteria : String
+ earnedAt : DateTime

Three-Layer Architecture

A clean view of what the user sees, what the app decides, and where data is kept

KidScribe is divided into three clear layers.

1. Presentation Layer — What users see

- **Login / Signup** — account access
- **Practice Screens** — stages, tracing, letters, words
- **Progress Screens** — progress, badges, children, classes

2. Business Logic Layer — What the app decides

- **Login Service** — handles login
- **Handwriting Check** — marks and feedback
- **Progress Service** — scores, stages, badges
- **Report Service** — parent and teacher reports

3. Data Access and Storage Layer — Where data is kept

- **Firebase Token** — login data
- **Saved Progress** — scores and streaks
- **Profiles & Classes** — children and class data
- **Templates + ML Kit** — assets and handwriting model

Flow: User screen → Service checks/updates data → Data is saved or read

This keeps the app organized because each layer has one job.

07

Important FRs from the SRS

FR-1:	The system shall let parents and teachers create a new account.
FR-2:	The system shall let parents and teachers log in to their account.
FR-3:	The system shall let parents and teachers create and manage child profiles.
FR-4:	The system shall let teachers create a class and add students.
FR-5:	The system shall let children practice basic strokes with a stylus.
FR-6:	The system shall let children practice writing letters from A to Z with a stylus.
FR-7:	The system shall check each written letter and show instant feedback.
FR-8:	The system shall let children practice writing common words.
FR-10:	The system shall guide children through five learning stages in order.
FR-14:	The system shall give badges, streaks, and rewards to children.
FR-18:	The system shall give teachers a class view of student progress.
FR-19:	The system shall track each child’s progress and show reports to parents.
FR-20:	The system shall save data on the device and sync it across devices.

08

Traceability in Practice

User Story	FR / Priority	Main Flow	Alternate Flow	Status
US-9: Letter writing	FR-6 / High	Child selects A-Z letter, sees guide, writes with stylus, and writing is saved.	Wrong writing shows feedback.	Done
US-10: Instant feedback	FR-7 / High	App checks writing and shows success for correct letter.	Wrong letter shows what to fix.	Done
US-14: Five stages	FR-10 / High	Stages are shown in order; next stage opens after completion.	Later stage stays locked.	In Progress
US-25: Parent report	FR-19 / High	Parent sees weekly progress, finished letters/words, and weak areas.	New child shows “no progress yet.”	Done
US-26: Offline practice	FR-20 / Medium	Child practises without internet and work is saved.	With no internet, data stays on device.	Done

09

The Qualities It Must Have

Key NFRs from the SRS:

NFR-01:	Result appears in about 5 second.
NFR-02:	Practice works without internet.
NFR-03:	Big buttons, few words, pictures, and sounds.
NFR-04:	Password is not stored in the app.
NFR-05:	Child handwriting stays on the device.
NFR-06:	Progress is saved after every check.
NFR-07:	Each child has separate progress.
NFR-08:	Works on phone/tablet, portrait/landscape.
NFR-09:	Three-layer design makes future changes easier.

These qualities protect the child experience: fast feedback, safe data, reliable saving, and easy use.

10

Current Project Status

29	14	1	14	68
user stories	done	in progress	to do	test cases

37
passing

KidScribe combines stylus-based handwriting practice, instant feedback, progress reports, rewards, and classroom support.

The project has a clear SRS base and can move forward by completing the remaining stories and improving test coverage.

PROJECT STRUCTURE

11

How the Codebase Is Organised

One folder per job, so the path of a file tells you what it may do

```
kidscribe/
  lib/  92 files  all the Dart code
    screens/  30  what the user sees
    blocs/  29  the rules and the state
    services/  15  storage, Firebase, ML Kit
    widgets/  7  pieces reused by many screens
    models/  4  plain data classes
    data/  3  letters, words, sentences
    theme/  2  colours and text styles
    main.dart  starts the app
    firebase_options.dart
  unit_testing/  19 files
    test/  16  129 test cases
  assets/  121 files
    art/  120  Fluent Emoji (MIT)
    sounds/  1
  android/  21 files  build config
  docs/  1  the SRS spreadsheet
  pubspec.yaml  packages and assets
```

build/ and .dart_tool/ excluded, as they are generated

A folder is a layer, not a feature

screens, blocs and services are the three layers. The arrows only point down: a screen never calls a service, and a service never knows a screen exists. The folder a file sits in says what it is allowed to touch.

Inside blocs, one folder per feature

20 of them: auth, learn, words, sentence, games, assign, tasks, badges, premium, sync and the rest. Everything about one feature's state is in one place.

Data is kept apart from code

models holds plain classes with no behaviour, data holds the letter shapes and word lists. Neither imports anything from the app, so both can be used by the tests directly.

12

What Travels Between the Parts

Four boundaries, and the exact shape of the data that crosses each one

Screen to Bloc
a method call

```
cubit.checkGuided(strokes)
```

plain Dart arguments

Bloc to Screen
an immutable state object

```
BadgesState(loading, earned, streak, best, practicedToday)
```

extends Equatable, so an unchanged state causes no redraw

Bloc to Service
a model in, a result out

```
TraceResult(score, stars, passed, coverage, precision, orderTip)
```

also FreehandResult(percent, passed, recognizedAs, kidMessage)

Service to device storage
one JSON string per key

```
letters_v2_<scope>  ->  {"A": {"guided": 3, "free": 82}}
```

SharedPreferences holds only text, so maps are encoded first

Service to Cloud Firestore
a typed map per value

```
{'t': 's'|'i'|'b'|'d'|'l', 'v': value}   in  kidscribe_sync/{uid}
```

the type tag survives the round trip, so a number never returns as text

Service to ML Kit
an Ink object

```
strokes of (x, y, t) points
```

the model reads the timing as well as the shape

The shared data classes: UserModel, ChildProfile, SchoolClass, TraceItem, Assignment, AssignItem, FeedbackNote, ScopeSummary
Each one carries toMap() and fromMap(), which is the only place a field name is written down twice.

One Request, End to End

A child finishes tracing the letter A and taps Check

1	UI	**The pen draws**	WritingPad keeps the strokes locally. The pen sends about 60 points a second, so none of it reaches a bloc.
2	UI	**The child taps Check**	The screen calls cubit.checkGuided(_gPad.strokes) and nothing else. It makes no decision of its own.
3	BLOC	**The cubit asks the marker**	LetterPracticeCubit calls TraceEvaluator.evaluate(drawn:, template:).
4	SERVICE	**The marker answers**	TraceResult(score: 88, stars: 3, passed: true, coverage: 0.96, precision: 0.91, orderTip: null)
5	BLOC	**The result is saved**	ProgressService.saveGuided('A', 3) keeps the best score only.
6	DATA	**Storage is written**	SharedPreferences key letters_v2_<scope> becomes {"A": {"guided": 3, "free": 82}}
7	BLOC	**Badges are worked out**	BadgeService.claimNewBadges() returns any badge earned just now, once only.
8	BLOC	**A new state is published**	emit(state.copyWith(guidedStars: 3, guidedResult: r, guidedTick: +1))
9	UI	**The screen reacts**	BlocBuilder redraws the stars. BlocListener sees guidedTick change and opens the result dialog exactly once.

Nine steps, and the screen appears in only two of them. Everything that decides anything sits in the middle layer, where the tests can reach it.

14

Why the Code Is Split This Way

Four rules decided where every class boundary went

Single responsibility

One class answers one question.

PasswordRules only knows what a good password is. SentenceChecker only knows the grammar rules. Neither touches storage or the screen, so each is tested on its own in milliseconds.

Layering

A class may only call the layer below it.

30 screens, 29 bloc files, 15 services. No screen imports a service. That one rule is what makes 58 of the 80 SRS tests runnable with no device attached.

Reuse

Anything used twice becomes its own class.

KidPage, KidHeader, KidCard and KidButton are used by every screen. WritingPad is shared by all four practice screens, so palm rejection was written once.

Testability by construction

A class is handed what it needs.

Every cubit takes its services as constructor arguments, so a test passes its own. currentUid() wraps Firebase so four services keep working when Firebase is not running.

The test suite is the proof: 129 test cases run in 23 seconds with no emulator, because the rules were never mixed into the screens.

The 22 tests that still need a real phone are the ones that genuinely do: Firebase sign-in, the handwriting model, notifications, the camera and two-device sync.

DIVISION OF CLASSES AND METHODS

How Big the Pieces Are

Measured over lib/ with a brace-matching parser, comments and strings stripped

92	18,255	224	675
Dart files	**lines in lib/**	**classes**	**methods**

Lines of code

	MAXIMUM	AVERAGE	MEDIAN
Per class	720	74.4	38
Per method	263	17.5	7

Largest class: _HomeView, 720 lines (home_screen.dart)
Largest method: _PennyPainter.paint, 263 lines (mascot.dart)

Why the large ones are acceptable

The median method is 7 lines, so most of the code sits in small pieces. Every method above 130 lines is either a Flutter build() or a CustomPainter.paint().

A build() is a declarative widget tree, not branching logic: it has one path through it. Splitting it into helpers moves lines around without lowering the complexity, and makes the layout harder to read.

paint() draws the mascot with straight-line canvas calls and no branching.

Where the rule was applied instead

The logic classes are deliberately small: SentenceChecker is 118 lines, PasswordRules 37, current_user.dart 19. TraceEvaluator, the one piece of real algorithm, is split into evaluate() and checkOrder() so each can be tested on its own. The size sits in the drawing code, not in the rules, which is where we wanted it.